

Playwright Testing Architecture

Detailed Technical Explanation & Reference Guide

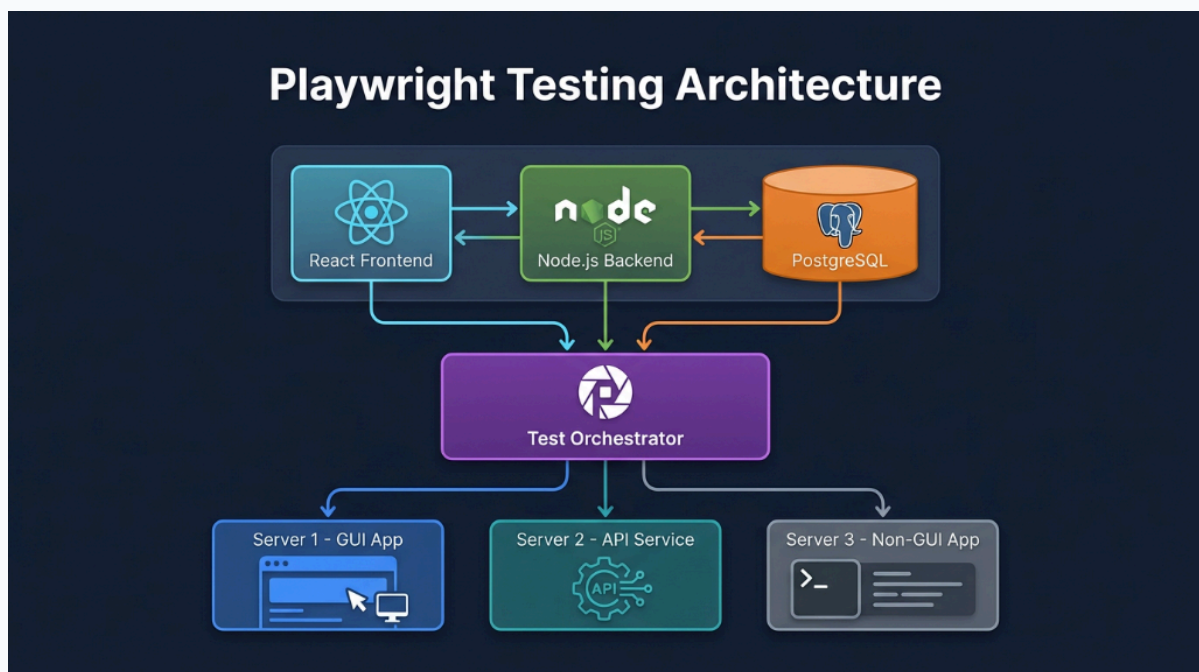


Figure 1: High-Level Architecture Overview

Table of Contents

- 1. The Big Picture
- 2. Top Layer - Application Stack
- 3. Middle Layer - Test Orchestrator
- 4. Bottom Layer - Server Types
- 5. Server Interactions
- 6. Communication Patterns
- 7. Configuration & Projects

- 8. Workers & Parallelization
- 9. Browser Connections (CDP)
- 10. HTTP Client for API
- 11. Database Interaction
- 12. Authentication & Headers
- 13. Multi-Environment Testing
- 14. Test Execution Flow
- 15. Error Handling & Debugging
- 16. Continuous Integration (CI)
- 17. Reporting & Artifacts
- 18. Common Testing Patterns
- 19. Best Practices
- 20. Scaling & Optimization

Section 1: The Big Picture

What is Playwright Test Orchestrator?

The Test Orchestrator is the central command center of your entire testing operation. Think of it as an air traffic controller that manages multiple planes (test workers) landing on different runways (servers). It doesn't actually do the flying—instead, it directs each plane to the right runway and monitors their progress.

The Architecture has Three Main Layers:

- **Top Layer:** Your actual application (React Frontend + Node.js Backend + PostgreSQL Database)
- **Middle Layer:** Playwright Test Orchestrator (the control hub)
- **Bottom Layer:** Three different server types receiving test traffic

Section 2: Top Layer - Your Application Stack

React Frontend (The User Interface)

This is what real users see and interact with. It's a web application running on your development machine or a server somewhere. The React Frontend:

- Displays buttons, forms, and content that users click on
- Makes requests to the Node.js Backend API when users perform actions
- Displays responses from the backend to the user
- Handles client-side logic and state management
- Lives at a specific network address (e.g., `localhost:3000` or `staging.app.com`)

Node.js Backend (The Logic Engine)

This is your server-side application that handles business logic and data processing. The Node.js Backend:

- Receives requests from the React Frontend
- Validates user input
- Executes business logic (calculations, rules, workflows)
- Communicates with the PostgreSQL database to read/write data
- Sends responses back to the Frontend
- Handles authentication and authorization

PostgreSQL Database (The Data Vault)

This is where all the persistent data lives. PostgreSQL:

- Stores user information, orders, products, transactions, etc.
- Receives queries from the Node.js Backend
- Returns data requested by the backend
- Maintains data integrity and consistency

How They Work Together (Real Example):

When a user registers for an account:

1. User fills out registration form in React Frontend (browser)

2. User clicks "Register" button
3. React Frontend sends user data to Node.js Backend via HTTP
4. Node.js Backend receives request, validates data, and inserts user into PostgreSQL
5. PostgreSQL confirms the data was saved
6. Node.js Backend sends success response to React Frontend
7. React Frontend displays "Registration successful!" message

This whole flow is what Playwright tests will verify.

Section 3: Middle Layer - Playwright Test Orchestrator

What is the Test Orchestrator?

The Test Orchestrator is Playwright itself—specifically the Playwright Test Runner that executes your tests. It's like a conductor of an orchestra who reads the score (your test files) and directs multiple musicians (worker processes) to play their parts in harmony.

Core Responsibilities:

1. Routing and Connection Management

The orchestrator decides which test goes to which server based on your configuration. You tell it:

- "Tests in the 'GUI' project should connect to `http://localhost:3000`"
- "Tests in the 'API' project should connect to `http://localhost:5000`"

2. Worker Process Management

The orchestrator creates multiple worker processes (usually matching your CPU cores). If you have a 4-core computer, workers 1 through 4 are

created. Each worker can run tests independently and simultaneously, making testing much faster.

3. Execution Control

The orchestrator manages how tests execute: timeout durations, retry logic for failed tests, and parallel vs. sequential execution strategies.

4. Environment Configuration

Loads global settings: Browser type (Chrome, Firefox, Safari), device emulation, network conditions, and recording settings.

5. Server Connection Management

Automatically starts servers (like `npm start`), waits for readiness (`webServer` config), and handles connection lifecycles.

6. Test Discovery & Reporting

Scans `.spec.ts` files, creates execution queues, collects results from all workers, and generates HTML reports.

Section 4: Bottom Layer - Three Server Types

Server 1: GUI Application (React Frontend)

This is your user-facing web application. When testing Server 1:

- Playwright launches a real browser instance.
- The browser connects to the React Frontend server.
- Your test code interacts with UI elements (clicks, inputs).
- Verifies UI display, animations, and client-side logic.

Characteristics: Requires browser, resource-intensive, tests UX/Visuals.

Server 2: API Service (Node.js Backend)

This is your backend application without a user interface. When testing Server 2:

- No browser is launched.
- Uses an HTTP client to send requests (GET, POST, etc.) directly.
- Verifies JSON responses, status codes, and business logic.

Characteristics: Fast, light on resources, tests logic/validation directly.

Server 3: Non-GUI Applications (Microservices)

Specialized services like Authentication, Payments, or Notifications.

Similar to API testing, these verify specific business domains in isolation or integration between services using HTTP requests.

Section 5: How The Three Servers Interact

The Real-World Flow

Imagine a user completing a purchase:

1. **Step 1 (GUI):** User interacts with React Frontend (Server 1) to add items and checkout.
2. **Step 2 (Frontend → Backend):** Frontend sends "Process Order" request to Node.js (Server 2).
3. **Step 3 (Backend Logic):** Backend validates stock and calculates taxes.
4. **Step 4 (Backend → Microservice):** Backend sends payment details to Payment Service (Server 3).
5. **Step 5 (Backend → Database):** Backend stores order in PostgreSQL.
6. **Step 6 (Response):** Backend confirms success to Frontend.
7. **Step 7 (GUI):** Frontend displays confirmation message.

What Playwright Verifies

Playwright can verify each part of this flow: GUI tests for the form, API tests for the validation logic, Microservice tests for payment processing, or Hybrid tests for the entire end-to-end flow.

Section 6: Test Orchestrator Communication Patterns

Pattern 1: Browser-Based GUI Testing

Test Orchestrator launches a browser. The browser acts as an intermediary, rendering HTML/JS and making requests to the backend. The test interacts with the visual layer.

Pattern 2: Direct API Testing

Test Orchestrator uses an HTTP client (no browser). It establishes a direct TCP connection to Server 2, sending raw HTTP requests and validating JSON responses. This is machine-to-machine communication.

Pattern 3: Hybrid Testing

Combines both approaches. A test launches a browser to interact with the GUI, but also makes direct API calls to the backend (to seed data or verify database state) or microservices.

Section 7: Configuration and Project Definition

A "Project" is a logical grouping of tests sharing the same configuration.

Project 1: GUI Testing

- **Name:** "Server1-GUI"
- **Target:** `http://localhost:3000`
- **Browser:** Chrome

- **Purpose:** Verify user interface.

Project 2: API Testing

- **Name:** "Server2-API"
- **Target:** `http://localhost:5000`
- **Browser:** None (HTTP client)
- **Purpose:** Verify backend API logic.

Project 3: Microservice Testing

- **Name:** "Server3-Auth"
- **Target:** `http://localhost:8001`
- **Browser:** None
- **Purpose:** Verify auth service specifically.

Section 8: Worker Processes and Parallelization

What is a Worker?

A worker is an independent OS process that runs tests. If you have 4 workers, you have 4 separate Node.js processes running simultaneously.

How Workers Are Assigned

The Orchestrator creates a queue of tests. Workers pick tests from this queue as they finish their previous task. This ensures load balancing across your CPU cores.

Isolation Between Workers

Worker 1's tests do not affect Worker 2. Each worker maintains its own isolated browser instance, cookies, and local storage. This prevents test pollution and flakiness.

Section 9: Browser Connections and CDP

Chrome DevTools Protocol (CDP)

Browsers are separate programs from Playwright. Communication happens via the Chrome DevTools Protocol.

The Connection Process:

1. Orchestrator launches browser process (e.g., Chromium).
2. Browser opens a network port (e.g., 9222).
3. Playwright connects via this port.
4. Playwright sends commands (Navigation, Click, Read).
5. Browser executes and returns responses.

Why Separate? Stability. If Playwright crashes, your tests stop, but if a browser tab crashes, the test framework can handle it. It also allows multiple browser versions to run simultaneously.

Section 10: HTTP Client for API Testing

An HTTP client sends requests without rendering visuals. It is significantly faster than browser testing.

Components:

- **Request:** URL, Method (GET/POST), Headers, Body (JSON).
- **Response:** Status Code (200, 404, 500), Headers, Body (Result Data).

Benefits: Fast execution, direct validation of business logic, ability to test edge cases and error handling that might be hard to trigger via UI.

Section 11: Database and Backend Interaction

Indirect Database Access

Tests usually do not connect directly to the database. Instead, they verify data via API endpoints.

Why Not Connect Directly?

- **Security:** DB might not be accessible from test runners.
- **Logic Check:** Tests need to verify the backend processed the data (logic layer), not just that it exists in storage.
- **Validation:** Direct DB access bypasses validation rules enforced by the API.

Note: Developers sometimes create special "test-only" API endpoints (e.g., `/api/test/seed-data`) to help setup or cleanup data quickly.

Section 12: Authentication and Headers

Handling Auth in Tests

1. Test requests login endpoint with credentials.
2. Backend validates and returns a Token (JWT).
3. Test saves this token.
4. Subsequent requests include the token in the `Authorization` header.

Token Reuse: Tests can login once (global setup), save the state/token, and reuse it across multiple tests to save time.

Section 13: Multi-Environment Testing

Playwright configurations can point to different environments (Local, Staging, Production) without changing test code.

- **Dev:** `http://localhost:3000`
- **Staging:** `https://staging.app.com`

- **Prod:** <https://app.com>

This allows you to write the test logic once and validate it across the entire deployment pipeline.

Section 14: Test Execution Flow

1. **Initialization:** Playwright reads config, spawns workers.
2. **Server Startup:** Launches local web server if configured.
3. **Browser Launch:** Starts browser instances via CDP.
4. **Test Execution:** Workers execute steps (Navigate → Action → Verification).
5. **Assertion:** Checks if results match expectations. Failures trigger screenshots/traces.
6. **Cleanup:** Contexts are closed, data cleared.
7. **Reporting:** Aggregates results into HTML report.

Section 15: Error Handling and Debugging

Common Errors

- **Connection Refused:** Server isn't running.
- **Timeout:** Server/Network too slow, or element didn't appear.
- **Element not found:** Selector is wrong or timing issue.
- **401 Unauthorized:** Auth token missing or expired.

Debugging Tools

- **Headed Mode:** Watch the test run visually.
- **Trace Viewer:** A "time-travel" debugger showing DOM snapshots, network requests, and console logs for every step.
- **Screenshots/Video:** Visual evidence of failures.

Section 16: Continuous Integration (CI)

In CI (GitHub Actions, Jenkins), tests run automatically on code push.

CI Differences:

- **Headless:** No visible UI runs to save resources.
- **Environment Variables:** CI injects secrets (API keys) and config (Base URLs) dynamically.
- **Artifacts:** Reports and traces are uploaded for developer review upon failure.

Section 17: Reporting and Artifacts

Playwright generates comprehensive artifacts to help understand test results:

- **HTML Report:** Full interactive dashboard of results.
- **Screenshots:** Images captured at the moment of failure.
- **Videos:** Full recording of the test session.
- **Traces:** Detailed logs of network, console, and DOM activity (most useful for debugging).

Section 18: Common Testing Patterns

- **Setup-Execute-Verify:** Create data → Do action → Check result.
- **Happy Path vs. Error Cases:** Testing both success scenarios and invalid inputs.
- **End-to-End (E2E):** Simulating a complete user journey across all systems.
- **Isolation Testing:** Verifying a single component or service independently.

Section 19: Best Practices

- **Keep Tests Independent:** No test should rely on another running first.
- **Use Meaningful Assertions:** Verify business outcomes ("Order Confirmed"), not just HTML implementation ("div exists").
- **Avoid Flaky Tests:** Use auto-waiting assertions instead of fixed sleeps.
- **Page Object Model:** Abstract UI interactions into separate classes for maintainability.

Section 20: Scaling and Optimization

Scaling Strategies

- **Sharding:** Split 100 tests across 4 machines (25 each) to run 4x faster.
- **Parallel Workers:** Maximize CPU core usage on a single machine.
- **Selective Testing:** Run only "Smoke Tests" on every commit, and full suite only on release.
- **Cloud Testing:** Offload execution to services like BrowserStack for massive scale.

Quick Reference Summary

Layer	Component	Purpose	Testing Method
Top	React Frontend	User Interface	Browser automation
Top	Node.js Backend	Business Logic	API testing
Top	PostgreSQL DB	Data Storage	Via API queries
Middle	Test Orchestrator	Control Hub	Configuration
Bottom	Server 1 (GUI)	Frontend App	Browser interaction

Layer	Component	Purpose	Testing Method
Bottom	Server 2 (API)	Backend Service	HTTP requests
Bottom	Server 3 (Non-GUI)	Microservices	HTTP requests

© Playwright Architecture Documentation. All rights reserved.